Tamamen Yerel RAG Sistemlerinde Vektör Veri Tabanı Performans Analizi ve Derin Araştırma Asistanı Geliştirilmesi

Mete Yağanoğlu Department of Computer Engineering, Faculty of Engineering, Ataturk University Erzurum, Turkey 0000-0003-3045-169X

Erhan Yüksel Turgut Department of Computer Engineering, Faculty of Engineering, Ataturk University Erzurum, Turkey 0009-0005-7455-5353

Uğurhan Daşdemir Department of Computer Engineering, Faculty of Engineering, Ataturk University Erzurum, Turkey 0009-0002-7981-3567

Hakan Enes Demir Department of Computer Engineering, Faculty of Engineering, Ataturk University Erzurum, Turkey 0009-0002-7714-2667

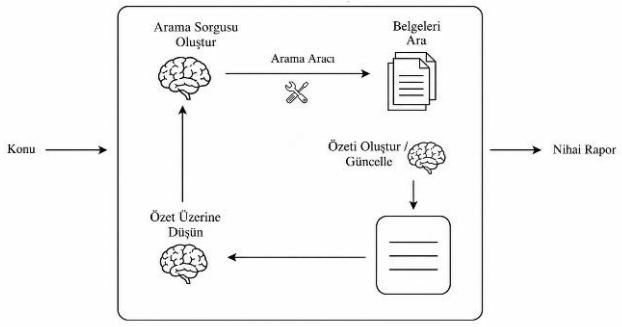
Abstract— In places like companies or institutions, finding fast and accurate information inside hundreds of documents is critical for decision-making and productivity. Regular keyword-based search algorithms just aren't good enough for this anymore, causing users to waste time digging through files to find the right source. On top of that, sharing sensitive documents with third-party cloud services creates huge security risks, especially since SaaS companies might use that private data to retrain their models. To solve these problems, this study introduces a two-stage, completely local AI search tool. First, we convert documents into semantic vectors and index them into vector databases using embedding models, which lets us use semantic matching instead of just basic keywords. Next, we use these vector databases to power an interactive conversation with a locally running large language model (LLM). If a user's search query isn't clear, the LLM actually asks clarifying questions to figure out their exact intention before generating a summary report and pointing to the source document. To find the best technical setup for this, we benchmarked five vector databases and 28 embedding models. Our results showed that the bge_squad model gave the highest accuracy with an nDCG@10 score of 0.9008 and a Recall@10 of 0.970. For system performance, ChromaDB handled the highest traffic at 3384.1 QPS, while Milvus was the fastest for indexing data at 5510.4 records per second. Because this entire system operates in a closed, internal network infrastructure, it completely eliminates the privacy risks of cloud-based AI, giving organizations secure, confidential, and direct access to their critical information while significantly boosting productivity.

Keywords— Retrieval-Augmented Generation (RAG), Semantic Search, Vector Database, Large Language Models, On-Premise AI, Air-Gapped Systems, Data Privacy, Embedding Models, Information Retrieval, Deep Research Assistant

I. INTRODUCTİON

Kurumsal ortamlarda geniş veri havuzları içerisinden doğru bilgiye hızlı bir şekilde erişmek, verimliliği ve karar alma süreçlerini etkilemektedir. Mevcut işleyiş genellikle TF-IDF ve BM25 gibi sözcük eşleştirme algoritmalarına dayanmaktadır. Ancak bu yöntemler, eş anlamlı kelimeler, farklı yazım şekilleri veya bağlama özgü ifadeler karşısında sözcüksel boşluk ya da kelime uyuşmazlığı sorunlarına yol açmaktadır [1,2]. Bu kısıtlamaları aşmak için kelimeleri ve dokümanları ortak vektör uzayına eşleyen semantik arama yaklaşımları geliştirilmiştir [1]. Vektör veri tabanları, birebir kelime eşleşmesi olmasa dahi anlamsal yakınlığa dayalı aramalar yapabilmeyi sağlar [3,4]. Semantik arama, RAG mimarisi ile birleştirilerek LLM'lerin özel verilere erişmesine ve cevaplarını kanıtlara dayandırmasına olanak tanır [3,5].LLM ve RAG sistemlerinin entegrasyonunda veri güvenliği önemli bir engeldir. Bulut tabanlı LLM kullanımı veri sızıntısı ve istem ifşası gibi güvenlik zafiyetleri barındırmaktadır [6]. Bu nedenle hassas dokümanların kurum içinde lokal bir altyapıda işlenmesi bir zorunluluktur [6].Bu çalışmada, bilgiye erişim zorluğunu aşmak amacıyla tamamen lokalde çalışan derin araştırma özellikli bir arama ve sohbet aracı önerilmektedir. Önerilen sistem, kurum içi dokümanları bir vektör veri tabanında indeksleyerek semantik arama gerçekleştirmekte ve kullanıcının isteğini akıllı bir asistan gibi anlamayı hedeflemektedir. Şekil 1'de detaylandırıldığı üzere, bu derin araştırma asistanı, kullanıcıdan gelen konuyu alıp arama sorgusu oluşturma, araç yardımıyla belgeleri arama, özet üretme ve bu özet üzerine tekrar düşünme adımlarını bir döngüde işleterek en doğru nihai raporu oluşturmaktadır.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE



Şekil 1 Derin araştırma (deep research) asistanının iteratif sorgu oluşturma, belge arama, özet güncelleme ve nihai rapor üretme akışını gösteren sistem şeması.

A. Motivasyon

Büyük Dil Modelleri, doğal dili anlama ve üretme konularında önemli başarılar elde ederek kurumsal ortamlarda bilgiye erişim ve karar alma süreçlerini dönüştürmüştür. Ancak bu modellerin kurum içinde kullanımı, veri gizliliği ve güvenliği gibi sorunları ortaya çıkarmaktadır.

Günümüzde gelişmiş LLM'lerin büyük bir kısmı üçüncü taraf sağlayıcılarca bulut tabanlı API'ler üzerinden sunulmaktadır. Kurumların gizli verilerini bu hizmetlere göndermesi veri sızıntısı, verilerin dışarı aktarımı ve istem enjeksiyonu gibi ciddi güvenlik zafiyetlerine yol açmaktadır.

Bu riskleri ortadan kaldırmanın en etkili yolu, veri işleme ve çıkarım süreçlerinin internetten izole edilmiş kurum içi yerel ağlarda çalıştırılmasıdır. Ancak modellerin yerel çalıştırılması, LLM'lerin güncel ve kuruma özel alan bilgisinden yoksun olması sorununu doğurmaktadır.

Bu bilgi eksikliğini veri gizliliğinden ödün vermeden çözmek için RAG mimarisi ön plana çıkmaktadır. RAG, kurumsal belgelerin yerel bir vektör veri tabanında indekslenerek LLM'e dinamik ve güvenli bir dış bilgi tabanı olarak sunulmasını sağlar.

B. Problem Tanımı

Geleneksel bilgi getirme sistemleri, TF-IDF ve BM25 gibi kelime eşleştirmesine dayalı yaklaşımlar üzerine kurulmuştur. Bu yöntemler hızlı olsalar da metinleri yalnızca bir "kelime çantası" olarak ele alarak bağlamı ve eş anlamlılık ilişkilerini göz ardı ederler.

Bu durum, kullanıcının bilgi ararken kullandığı terimler ile dokümandaki terimlerin farklı olduğu durumlarda sözcüksel boşluk problemine yol açar. Anahtar kelime tabanlı sistemler, eş anlamlılık, çok anlamlılık ve doğal dil karmaşıklığı durumlarında başarısız olabilmektedir. Bu yetersizlikleri aşmak için sorguları ve dokümanları yüksek boyutlu vektör uzaylarına eşleyen semantik arama ve yoğun geri getirim modelleri ön plana çıkmıştır. Ancak RAG mimarilerinde semantik arama, yerel sistemler için yüksek hesaplama maliyeti ve performans problemleri doğurmaktadır.

Bu nedenle, yerel bir RAG sisteminde en uygun açık kaynaklı embedding modelinin ve en performanslı vektör veri tabanının seçilmesi önemli bir mühendislik ve optimizasyon problemidir.

C. Katkılar

Bu çalışmanın temel amacı, bulut tabanlı sistemlerin veri gizliliği zafiyetlerini ve geleneksel arama yöntemlerinin anlamsal boşluk yetersizliklerini aşan, güvenli ve yüksek performanslı bir bilgi getirim sistemi geliştirmektir.

Tamamen Yerel ve Modüler RAG Sistemi ile dış ağ bağlantılarına bağımlılığı ortadan kaldıran, kurum içi yerel ağlarda çalışan güvenli bir RAG mimarisi tasarlanmıştır. Bu sistem, hassas kurumsal verileri dışarı sızdırmadan işleyerek veri egemenliğini güvence altına almaktadır.

Çoklu vektör veri tabanı benchmark altyapısı ile Milvus, Qdrant, ChromaDB, Weaviate ve LanceDB gibi farklı vektör veri tabanlarının performansı karşılaştırılmıştır. Bu karşılaştırma, veri ekleme süresi, QPS, gecikme süresi ve recall gibi metrikler üzerinden yapılmıştır.

Etkileşimli ve çapraz platform derin araştırma asistanı arayüzüyle kullanıcıların doküman yüklemesi, LLM seçmesi ve doğal dil sorgularıyla bilgiye erişmesi sağlanmıştır. Asistan, anlamsal aramadan dönen sonuçları sentezleyerek kaynağı belli, doğrulanabilir ve bağlamı güçlendirilmiş cevaplar sunmaktadır.

II. LİTERATURE REVİEW

A. Erişim Artırımlı Üretim (RAG)

Retrieval-Augmented Generation (RAG) mimarisi 2020 yılında Lewis ve arkadaşlarının Facebook AI'da sunduğu retrieve-then-read paradigmasından ortaya çıkmıştır [1]. RAG süreci temel olarak üç aşamadan oluşur. Bu aşamalar Retrieval, Augmentation ve Generation'dir. Retrieval, kullanıcı bir soru sorduğunda bu soruyu bir vektöre dönüştürür ve vektör veri tabanında ilgili dokümanları arar. Augmentation, bulunan ilgili metin parçalarını kullanıcının sorgusuyla birleştirir. Generation, LLM modeline yeni bir bağlam sunar ve LLM bu bağlam ile sonucu oluşturur. RAG mimarisinin sunduğu bazı avantajlar şunlardır, Model önüne koyulan somut kaynaklar ile halüsinasyon sorunu yaşamaz. Model eğitilme tarihi eski olsa da veri tabanına güncel belgeler eklenerek modelin güncel verilere ulaşımı sağlanır. Şirketler ve kurumlara özel sadece kendi dokümanlarını içeren gizlilik ve güvenlik sağlayarak çalışabilirler. Bununla birlikte RAG mimarisinin bazı sınırlılıkları da mevcuttur. Retrieval kalitesi düşük olduğunda üretilen yanıt da hatalı olabilir. Büyük belge koleksiyonlarında vektör araması maliyetli hale gelir. Ayrıca chunk boyutu ve overlap gibi hiper parametrelerin manuel ayarlanması gerekmektedir [3].

RAG'ın temel bileşenlerinden Embedding modeller metinleri vektörlere dönüştürür. Bu vektörlerin saklanması için vektör veri tabanları kullanılır. Vektör veri tabanları yüksek arama hızlarına sahiptir. Ayrıca pipeline organizasyonu için LangChain veya LlamaIndex gibi orkestrasyon

framework'leri kullanılır. Bu framework'ler belge yükleme, parçalama, retrieval ve üretim adımlarını uçtan uca yönetir.

Naive RAG, belgeleri sabit boyutlu parçalara böler ve dense embedding ile indeksleyerek en yakın k parçayı getirir. Getirilen bu parçalar LLM'e iletilir. Naive RAG mimarisi NLP görevlerinde düşük precision/recall, lost-in-the-middle etkisi gibi sınırlılıkları nedeniyle yetersiz kalmaktadır [3].
Bu problemlere çözüm olarak geliştirilen Advanced RAG, sorgu preprocessing ve post-retrieval optimizasyonlarını ekler. Gao ve arkadaşlarının önerdiği HyDE yaklaşımı, normal retrieval'daki sorgunun doğrudan embed edilmesinin aksine bir yöntem kullanır [7]. Sorguyu aldıktan sonra LLM ile bir cevap belgesi oluşturur. Oluşturulan bu belgeyi embedding yapar ve vektör araması yapar. Bu yöntem ile direkt sorgu yerine belge ile arama yapılarak daha fazla semantik yakınlık yakalanır. Hibrit aramada ise iki farklı arama yöntemini beraber kullanır. BM25, tam kelime eşleşmesine dayanır. Dense Embedding ise semantik benzerliğe dayanır. Bu aramalardan sonra RRF skoru ile bulunan belgelerin benzerlikleri hesaplanır ve en ilgili belgeler bulunur. RAG-Fusion ise bu hibrit arama mantığını bir adım ileriye taşır [6]. Orijinal sorgudan birden fazla sorgu varyantı üretilir. Her varyant için ayrı retrieval yapılır. Elde edilen sonuçlar RRF ile birleştirilir. Böylece tek bir sorgunun kaçırabileceği belgeler farklı varyantlar aracılığıyla yakalanır.

HyDE ve hibrit arama aşamalarından sonra elde edilen belge listesi Cross-encoder ile son kez düzenlenir. Retrieval aşamasında hız ön planda olduğu için bu sebeple hassasiyette kayıplar olmaktadır. Bu aşamada Bi-encoder kullanılır. Bi-encoder, sorgu ve belgeyi ayrı ayrı embed eder ve sonra benzerliklerini hesaplar. Hız avantajı vardır fakat bağlam kaybolur. Bu aşamadan sonra elde edilen top n belgeye Cross-encoder uygulanır. Cross-encoder, sorgu ve belgeyi birlikte modele verir ve aralarındaki ilişkiyi doğrudan öğrenir. Daha hassastır fakat yavaştır. Bu sebeple küçük bi-encoder'dan sonraki küçük belge kümesine uygulanır [6].

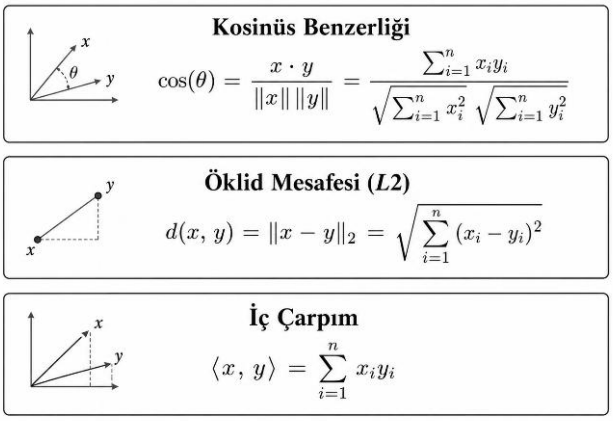
Advanced RAG bu optimizasyonlarla Naive RAG'ın sınırlılıklarını büyük ölçüde giderse de pipeline yapısı hala sabittir. Her sorgu aynı modüllerden aynı sırayla geçmek zorundadır. Farklı görevler farklı pipeline kombinasyonları gerektirdiğinde bu sabitlik yetersiz kalmaktadır [4].

Mimari gelişiminin en kapsamlı aşaması Modular RAG'dır. Gao ve arkadaşları tarafından 2024 yılında önerilmiştir [4]. Bu mimari ile beraber Naive RAG'ın sabit mimarisi yerine, istenilen ekleme ve çıkartmaların yapılabildiği bağımsız modüllerin birleşimi ile bir araya gelen bir mimari oluşturulmuştur. Bu mimari dört temel akış örüntüsü tanımlar. Linear örüntü, Naive RAG'ın doğrusal retrieve-generate akışına karşılık gelir. Conditional örüntü, sorgu karmaşıklığına göre pipeline'ın dallandığı yapıdır. Branching örüntü, paralel retrieval yapılan RAG-Fusion gibi yaklaşımlara karşılık gelir. Looping örüntü ise Self-RAG gibi iteratif retrieve-generate döngüsünün kullanıldığı yapıdır. Naive ve Advanced RAG bu dört örüntü içinde yalnızca Linear örüntünün özel durumları olarak yeniden konumlandırılır [4].

Search modülü ile hangi kaynakların (Vektör DB, SQL Veri tabanları, Web Araması vb.) kullanılacağı belirlenir. Sorgu türüne göre birden fazla kaynak birlikte kullanılabilir. Memory modülü, geçmiş retrieval'ları ve verilen sonuçları hatırlar. Aynı belgeyi tekrar tekrar getirmek yerine cache'i kullanır. Routing modülü ile sorgunun karmaşıklığına göre hangi pipeline'ın çalışacağı belirlenir. Örneğin basit bir soru retrieval yerine direkt LLM'e gider. Karmaşık bir sorgu ise iteratif retrieval döngüsüne yönlendirilir. Bir diğer modül Predict'tir. Bu modül ile retrieval yapmak yerine LLM'in parametrik bilgisi kullanılır. Yani bazı sorgular için modelin kendi bilgisi yeterlidir ve dış kaynak aramaya gerek yoktur. Fusion modülü ile birden fazla retrieval sonucu birleştirilir. RAG-Fusion'daki RRF mantığı da bu modülün içerisinde çalışır. Farklı sorgu ya da kaynaklardan gelen sonuçlar burada birleştirilir. Scheduling ise modüllerin hangi sırada ve ne zaman çalışacağını yönetir.

B. Vektör Veri Tabanı Yönetim Sistemleri

Günümüzde veri yönetimde metin, görüntü ve ses gibi verilerin yüksek boyutlu sayısal vektörler olarak temsil edilmesi, semantik aramanın temelini oluşturmaktadır. Geleneksel ilişkisel veri tabanı yönetim sistemleri kesin eşleşmelere dayalı yapılandırılmış tablolar için tasarlanmışken, Vektör Veri tabanı Yönetim Sistemleri, bu yüksek boyutlu vektörleri verimli bir şekilde depolamak, yönetmek ve anlamsal yakınlıklarına göre sorgulamak için özel olarak geliştirilmiş yazılımlardır. Vektör Veri Tabanı Yönetim Sistemleri (VDMS), sorgu vektörü ile veri tabanındaki vektörler arasındaki anlamsal yakınlığı ölçmek için Kosinüs Benzerliği, Öklid Mesafesi (L2) veya İç Çarpım gibi matematiksel mesafe metriklerini (Bkz. Şekil 2).



Şekil 2 Vektörler Arasındaki Anlamsal Yakınlığı Ölçme Formülleri

Vektör uzaylarında boyut sayısı arttıkça veri noktaları arasındaki mesafe hesaplamaları donanımsal olarak son derece maliyetli hale gelir. Bu maliyet nedeniyle, tüm veri tabanının sırayla tarandığı kaba kuvvet (brute-force k-NN) aramaları gerçek zamanlı sistemler için kullanışsızdır. Bu darboğazı aşmak için VDMS, latency ile recall arasında optimum bir denge kurmayı sağlayan k-NN indeksleme algoritmalarından faydalanır. Literatürde en sık kullanılan vektör indeksleme teknikleri şunlardır:

Grafik tabanlı indeksler (örn:HNSW) vektörleri hiyerarşik ve gezinilebilir bir grafik ağı olarak yapılandırarak logaritmik arama hızları sunar ve mevcut en yüksek doğruluğu sağlar. Ancak bellek (RAM) tüketimi oldukça yüksektir.

Kümeleme ve sıkıştırma tabanlı indeksler (örn. IVF- Inverted File, PQ- Product Quantization) vektörleri K-means gibi algoritmalarla kümelere ayırır ve bellek alanından tasarruf etmek için vektörlerin boyutlarını alt uzaylara bölerek sıkıştırır (quantization). Bu sayede büyük ölçekli veri setleri kısıtlı donanımlarda bile aranabilir.

Günümüzde VDMS'ler mimari olarak iki ana kategoriye ayrılmaktadır. Vektör işlemlerine özgü olarak sıfırdan tasarlanmış ve yüksek verim odaklı yerel sistemler (ör. Milvus, Qdrant, Pinecone, ChromaDB) ve mevcut veri tabanlarına vektör yetenekleri entegre edilmiş genişletilmiş sistemlerdir (ör. PostgreSQL tabanlı pgvector veya Elasticsearch).

Kapalı ağda çalışan bir RAG mimarisi tasarlarken doğru VDMS'nin seçimi, sistemin saniyedeki sorgu kapasitesi (QPS), indeksleme süresi ve RAM/GPU kullanım optimizasyonunu doğrudan etkileyen en kritik kararlardan biridir.

C. Bilgi Erişiminde Performans Kıyaslaması

Bilgi erişim sistemlerinde başarım ölçümü, bir veri tabanı, arama motoru, vektör veri tabanı veya RAG sisteminin kullanıcı sorgusuna cevap olarak verdiği yanıtların etkinliğini değerlendiren bir işlemdir. Geleneksel olarak bu işlem, sistemin alaka düzeyini, hızını ve güvenirliğini değerlendiren bir dizi ölçüt üstünden yürütülmektedir.[8] Modern sistemlerin, RAG mimarilerinin ilerlemesiyle beraber, bu ölçütler sadece döküman bulma başarımı değil, verilen yanıtın doğruluğu ve sistemini verimliliğini de kapsayan çok boyutlu bir oluşuma dönüşmüştür.

Geleneksel sıralama ve erişim metirkleri modern bilgi erişim sistemlerinde, temel metrik olarak “Precision” ve “Recall” değerleri yeterli değildir. Bu metrikler sonuçların sıralamadaki konumlarını dikkate almazlar. Bu sebeple modern çalışmalarda aşağıdaki gelişmiş metrikler temel alınmaktadır.

Mean Average Precision (MAP) sisteminin döndürdüğü tüm ilgili sonuçların sıralamadaki konumlarını dikkate alarak başarımı ölçmektedir. Özellikle çoklu doküman sentezi gerektiren görevlerde varsayılan metrik olarak kabul edilir. MAP, sadece ilk ilgili sonuca bakmak yerine tüm sıralamaya bakarak değerlendirme yapar [9]. Mean Reciprocal Rank (MRR) ise sisteminin ilgili ilk sonucu ne kadar hızlı bulduğunu değerlendirir. Özellikle tek bir cevabın istenildiği soru-cevap görevleri için önemli bir başarım göstergesidir [10]. Normalized Discounted Cumulative Gain (nDCG) ise sistemin sonuçlarının sadece alakalı alakasız olmalarına değil, ilgi düzeylerinin derecesine yoğunlaşır. İlgili sonuçların alt sıralarda yer almasını logaritmik olarak cezalandırarak, en yararlı bilgi penceresinin üst sıralarda yer almasını sağlar [11].

RAG Üçlüsü (The RAG Triad) ve Üretim Kalitesi RAG sistemlerinde başarım, retriever, generator birimleri arasındaki etkileşimi kontrol eden ve literatürde The RAG Triad olarak bilinen 3 ana modüle dayanır. Context Relevance, sistemin döndürdüğü dokümanların sonucun kullanıcı sorgusuyla ne kadar uyuştuğunu ölçerek getirici modülün gürültülü veri miktarını denetler. Groundedness, üretilen cevabın getirilen bağlama ve kaynak dokümanlara bağlı olup olmadığını, modelin halüsinasyon görüp görmediğini denetler [12]. Answer Relevance, üretilen cevabın kullanıcı sorusuna ne kadar doğrudan ve eksiksiz yanıt verildiğini değerlendirir.

Bu değerlendirmeler çoğunlukla RAGAS, TruLens veya DeepEval gibi LLM-as-a-judge temelini kullanan modern çevrelerce tercih edilir.

Sistem verimliliği ve operasyonel performans bilgi erişim sistemlerinin çıktı kalitesi kadar sistem kaynaklarının kullanımlıda önemli bir kıyaslama standartlıdır. Önemli metrikler Latency, Throughput ve kaynak kullanımıdır. Latency (Gecikme) ilk token üretim süresi (TTFT), token başına üretim hızı (TPOT) ve uçtan uca toplam yanıt süresi (E2E Latency) olarak raporlarınır [13]. Throughput (İş Hacmi), sistemin birim zamanda işleyebileceği sorgu sayısı (QPS) veya token işleme kapasitesidir. Kaynak kullanımı, CPU/GPU kullanımı, VRAM tüketimi ve I/O bant genişliği gibi donanımsal metrikler, sistemin ölçeklenebilirliği acısından testlere dahil edilir [14].

III. MATERİAL AND METHOD

Modern bilgi erişim sistemlerinde modülerlik, esneklik, ölçeklenebilirlik ve bileşen bazlı performans metrikleri analizi ihtiyacıyla ortaya çıkmıştır. Önerilen sistem mimarisi, geneksel "getir ve üret" (retrieve-then-generate) şemasını, bağımsız ve değiştirilebilir oluşan bir modüler bir yapı yaklaşımıyla yeniden kavramsallaştırmaktadır. Bu tasarım yeniliği, operasyonel RAG sistemi ile kapsamlı bir bechmark yapısının tek bir hibrit yapıda kullanılmasıdır.

RAG hattının ayrıştırılmasında, sistem merkezinde RAG aşamalarının birbirinden bağımsız 5 temel sistem modülüne ayrışmasında yer almaktadır. Bu yapı çevresinde veri işleme ve embbeding modülü, ham metinleri anlamsal vektör temsillerine dönüştürerek vektör veri tabanına aktarılacak veri yapısını oluşturur. İndeksleme ve vektör depolama modülü, Milvus, Qdrant ve Chroma gibi farklı vektör veritabanları üzerinde yapılacak benchmark testleri sonucunda performans, gecikme süresi ve doğruluk açısından en iyi skoru veren sistemi belirlemeyi amaçlar. Erişim modülü, vektör benzerliği ile anahtar kelime eşleştirmesini birlikte kullanan hibrit stratejileri kullanarak ilgili belgeleri getirir. Yeniden sıralama modülü, sonuçları daha hassas modellerle değerlendirerek sıralama kalitesini iyileştirir. Son olarak üretim modülü, Ollama üzerinden Llama, Gemma gibi farklı büyük dil modeli arka uçlarını kullanarak nihai yanıtı oluşturur.

A. Veri Toplama ve İşleme

RAG sistemlerinin başarısı verinin kalitesi ve işlenmesine bağlıdır. PDF belgeler, makaleler ve kurumsal raporlar gibi yoğun bilgi içeren kaynak türüdür. PDF belgeleri yapısı gereği metin, tablo ve görseller gibi veri türlerini bir arada bulundururlar. Bu durum metin çıkarımını zorlaştırmaktadır. Çünkü belge düzeni anlamsal bir hiyerarşi içindedir. Başlıklar, alt başlıklar, paragraflar ve tablolar birbirinden ayrıştırılarak işlenmelidir. Aksi halde retrieval aşamasında anlamsız chunklar elde edilir. Düz metin çıkarma yöntemleri yerine ayrıştırmalı yöntemler kullanılarak retrieval kalitesi artmaktadır.

Chunking stratejileri RAG sisteminin performansı üzerinde doğrudan belirleyici bir etkiye sahiptir. Naive RAG'da yaygın olarak kullanılan fixed size chunking, belirli bir token sayısında ulaşıldığında metni böler. Bu yöntemin uygulanması kolaydır fakat cümle ve paragrafların sınırlarını gözetmez. Bu durum anlam bütünlüğünün bozulmasına sebep olabilir. Cümlelere göre parçalama yapmak kısmen çözüm olabilir ama yine de yetersiz kalır. Hiyerarşik chunking ise belgenin hiyerarşik yapısını temel alır. Belge bölümlere, paragraflara ve en sonunda cümlelere ayrılır. Retrieval aşamasında daha alt birimlerde arama yapılırken LLM'e daha geniş üst birimler iletilir. Bu yaklaşım small to big retrieval olarak da adlandırılmaktadır ve teknik dökümanlarda bağlam kaybını azaltmaktadır.

Meta verileri eklenerek chunklara belge adı, başlık ve sayfa numarası gibi meta veriler eklenir. Bu veriler retrieval sonrası filtreleme ve sıralama için kullanılır. Alakasız parçalar bu yöntem ile elenir. Sonuç olarak retrieval aşamasına sadece anlamlı parçalar taşınır.

B. Çok Modelli Embedding Katmanı

Yerel RAG mimarilerinde anlamsal aramanın başarısı, yapılandırılmamış metinleri yüksek boyutlu yoğun vektörlere dönüştüren embedding modellerinin kalitesine doğrudan bağlıdır. Literatürde BERT, RoBERTa ve DeBERTa gibi farklı Transformer tabanlı mimariler, metinlerdeki anlamsal ve sözdizimsel ilişkileri farklı dikkat mekanizmalarıyla yakalamaktadır. Tek bir gömme modeline bağlı kalmak yerine birden fazla modelin entegre edilmesi, metin temsilinin kalitesini artırarak farklı dilsel bağlamların daha güçlü bir şekilde yakalanmasına olanak tanır. Bu doğrultuda geliştirilen sistemde all-MiniLM-L6-v2 Snowflake Arctic, BGE M3 ve Qwen3.5 Embeding türevleri gibi farklı açık kaynaklı dil modellerini destekleyen modüler bir çoklu model gömme katmanına sahiptir.

Çoklu model kullanımının ve tamamen yerel çalışmanın getirdiği en büyük donanımsal zorluk, sınırlı grafik işlemci bellek (VRAM) kapasitesidir. Embedding modellerinin ve üretimden sorumlu LLM'lerin aynı anda GPU üzerinde barındırılması, ciddi bir kaynak çekişmesine ve yetersiz bellek hatalarına yol açmaktadır. Bellek darboğazını aşmak için tüm embedding yükünü CPU'ya devretmek bir seçenek olsa da bu durum çok daha yüksek çıkarım gecikmelerine neden olarak RAG pipeline'nın genel performansını düşürmektedir.

Bu çalışmada, hız ve bellek kısıtlamaları arasında optimum bir denge kurmak amacıyla dinamik GPU/CPU yönetim stratejisi geliştirilmiştir. Bu stratejiye göre, kısıtlı VRAM'e sahip standart donanımlarda bile sistemin kesintisiz çalışabilmesi için gömme modelleri varsayılan olarak sistemin ana belleğinde atıl durumda tutulmaktadır. Sisteme bir doküman yüklendiğinde veya kullanıcıdan bir sorgu geldiğinde, vektörizasyon işlemi için seçilen model geçici olarak GPU'ya aktarılır. Hızlandırılmış GPU donanımı kullanılarak vektörlerin hesaplanması tamamlandığı anda model tekrar CPU'ya alınır ve GPU belleği özel komutlarla (ör. torch.cuda.empty_cache) programatik olarak temizlenir.

Geliştirilen bu VRAM dostu kaynak yönetim mimarisi, büyük veri setlerinin (PDF vb.) indekslenmesi ve parçalanması sırasında sistemin çökmesini engellerken, farklı dil modelleri arasında geçiş yapılırken donanım darboğazı yaşanmasının da önüne geçmektedir. Böylelikle, modüler yapıdaki çoklu model embedding katmanı, kısıtlı donanımlarda dahi hem yüksek hızda vektörizasyon yapabilirken hem de kesintisiz RAG operasyonu sunabilmektedir.

C. Sorgu Netleştirme ve İyileştirme

Bilgiye erişim ve konuşma dayalı sistemlerde, kullanıcıların isteklerini genellikle eksik, belirsiz veya çok yönlü belirtmeleri, sistem başarını etkileyen en önemli güçlüklerden biridir [15]. Bu durum, kullanıcının isteğinin doğal dille anlatışı ile arama araçlarının sorgu beklentisi ile arasındaki semantik boşluk olarak tanımlamaktadır [16]. Bu kısıtlamayı aşmak için geliştirilen LLM yardımlı sorgulama araçları, pasif bir getir-oku sisteminden çıkarak, kullanıcıyı proaktif olarak süre katan geri bildirim odaklı bir yapı sunmaktadır.

Akademik literatürde, niteleyici sorgulama genellikle 3 aşamada incelenmektedir, netleştirme ihtiyacının tahmini, uygun sorunun oluşturulması ve kullanıcı memnuniyetinin değerlendirilmesi [17]. En gelişmiş yaklaşımlardan biri olarak kabul gören AsK(Aspects and Retrieval based Hybrid Clarification), sorgudaki belirsizlik seviyesini kontrol ederek etkin bir karar verme stratejisi uygular [15]. Eğer kullanıcı isteği yetersiz veya belirsizse, sistem geniş alan bilgilerini (domain aspects) kullanarak soru sorar. Belirsizlik daha düşükse getirilen en iyi sonuçlar (top-k) arasından niteleme yapar [15]. Aynı tarzda ICR (Iterative Clarification and Rewriting) netleşmeme sorularını birer dönüm noktası olarak kullanarak yeniden yazar [18]. Bu süreçte LLM, konuşma geçmişindeki zamirleri ve atlanmış kelimeleri analiz ederek zayıf sorguyu, bilgi erişim sistemleri için optimize edilmiş ve bağlamdan bağımsız kapsamlı bir sorguya dönüştürür. LLM'lerin doğal dil anlama yetenekleri sayesinde, niteleyici sorular kullanıcının bilgi açığını kapatmaya odaklanır ve elde edilen geri bildirimler anında sistemin üretim modüllerine entegre edilir

D. Yerel LLM Orkestrasyonu

Yerel RAG mimarileri, bilgi getirme sürecini ve dil modelinin üretken kapasitesini senkronize bir şekilde kontrol etmek amacıyla merkezi bir mekanizmaya ihtiyaç duymaktadır. Bu çalışmada, sistemin API katmanı olarak

hızlı, asenkron ve yüksek performanslı bir Python web çerçevesi olan FastAPI kullanılmıştır. FastAPI PDF dosyalarının yüklenmesi, parçalara ayrılması ve indekslenmesi, anlamsal aramaların gerçekleştirilmesi ve dil modeliyle sohbet edilmesi gibi tüm işlemleri birbirinden bağımsız uç noktalar üzerinden modüler bir şekilde yönetmektedir.

LLM'lerin yerel donanımlarda bellek açısından verimli bir şekilde çalıştırılması ve yönetilmesi görevini ise Ollama üstlenmektedir. Kullanıcılar, Flutter arayüzü üzerinden donanım kapasitelerine göre nemotron-3-nano:4b, medgemma1.5, granite4.1:3b ve ministral-3:3b gibi farklı modeller arasında dinamik olarak geçiş yapabilmektedir. FastAPI tabanlı orkestratör, kullanıcıdan gelen bu seçimi algılayarak Ollama servisine ilgili modelin yüklenmesi ve çıkarım yapması için talimat verir.

Arayüz, kullanıcı sorgularının karmaşıklığına ve bilgi ihtiyacına göre iki farklı operasyonel modda çalışacak şekilde yapılandırılmıştır.

Normal Sohbet Modu Kullanıcının genel geçer bir soru sorduğu durumlarda, sistem vektör veri tabanında gereksiz bir arama yapmak yerine sorguyu doğrudan seçili LLM'e ileterek standart bir sohbet akışı sağlar. Bu durum, hesaplama kaynaklarının (VRAM) gereksiz kullanımını engeller.

Derin Araştırma Modu (Deep Research) Sistem, dış bir bilgi kaynağına ihtiyaç duyduğunda veya analiz odaklı bir istek geldiğinde bu modu tetikler . Bu modda FastAPI , ilk olarak kullanıcının sorusuyla vektör veritabanı (örn. ChromaDB) üzerinde anlamsal arama (similarity search) gerçekleştirir. Veri tabanından dönen en alakalı belge parçaları (top-k chunk) alınır ve sistem, modeli basit bir yanıt yerine kapsamlı bir özet raporu üreten özel bir istem (prompt) ile çalıştırır.

Bu dinamik modüler yönlendirme ve sistem yeteneği sayesinde asistan, hem standart bir sohbet botu esnekliği sunmakta hem de kurumsal belgeler üzerinde doğruluk payı yüksek ve halüsinasyondan arındırılmış derinlemesine araştırma yapabilen uzman bir sisteme dönüşmektedir.

E. Flutter Tabanlı Kullanıcı Arayüzü Tasarımı

RAG tabanlı sistemlerin son kullanıcıya sunulmasında arayüz tasarımı kritik bir rol üstlenir. Flutter, tek bir kod tabanından farklı platformlara derleme yapılmasına olanak tanıyan çapraz platform bir UI framework'ü olarak RAG sistemlerinin tutarlı biçimde sunulmasını mümkün kılar. Arayüz tasarımında sorgu girişi, akış tabanlı yanıt gösterimi ve kaynak paneli olmak üzere üç temel bileşenin bir arada düzenlenmesi gerekir. Kurumsal kullanım senaryolarında oturum bazlı raporlama, sorgu-yanıt çiftlerinin zaman damgasıyla kaydedilmesi ve dışa aktarılabilir formatlarda sunulması temel işlevler arasında yer alır. Şeffaf kaynak izlenebilirliği ise RAG sistemlerini geleneksel LLM uygulamalarından ayıran en önemli özelliklerden biridir; üretilen her yanıt için hangi belgeden ve hangi sayfadan geldiği referans bağlantıları aracılığıyla kullanıcıya sunulur. Özellikle hukuki, tıbbi veya akademik senaryolarda her kaynak pasajının retrieval skorunun görselleştirilmesi şeffaflığı zorunluluk haline getirmektedir [7].

IV. RESULTS

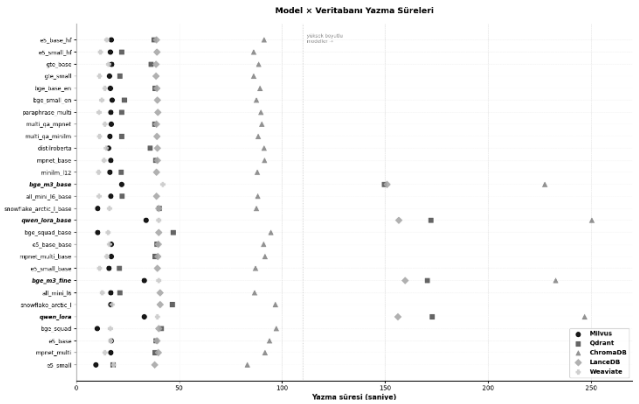
Vektör veri tabanları ile RAG sistemlerinde embedding vektörlerinin depolanması ve sorgulanması sağlanmaktadır. Her veri tabanının mimari farklılıkları doğrudan okuma ve yazma performansına etki etmektedir. Bu çalışmada ChromaDB, Milvus, Qdrant, LanceDB ve Weaviate olmak üzere beş farklı vektör veri tabanı karşılaştırılmaktadır. 28 farklı embedding model üzerinde yazma süresi kriterine göre karşılaştırma yapılmıştır. Temel kriter olarak yazma süresinin seçilmesinin nedeni RAG sistemlerinde ingestion aşamasının tüm pipeline'ın başlangıç maaliyetini belirlemesidir. Yazma süresi yüksek olursa büyük belgelerde indeksleme süreci uzatmakta ve sistemin hazır olma süresini geciktirmektedir [19].

Benchmark sonuçlarına göre Milvus tüm embdedding modellerinde en düşük yazma sürelerini elde etmiştir. Milvus, e5_small modeliyle 9,497 saniye, bge_squad modeliyle 10,25 saniye yazma süresi elde etmiştir. Bu sonuçlar Milvus'un dağıtık mimarisinin ve HNSW tabanlı indeksleme yapısının yazma performansına avantaj sağladığını ortaya çıkartmaktadır. İkinci en iyi performansı Weaviate sergilemiştir. Weaviate, minilm_l12 modeliyle 10,81 saniye ve all_mini_l6_base modeliyle 11,006 saniye yazma süresi elde etmiştir. Bu iki veri tabanının performansları birbirine yakın olsa da Milvus üstünlüğünü korumaktadır.

Qdrant'ın performansı Milvus ve Weaviate'in gerisinde kalmıştır. Yazma süreleri 17,86 - 46,972 saniye arasında değişmektedir. Qdrant, kullanılan embedding modeline göre farklı performans vermektedir. LanceDB ise 37,926 - 40,692 saniye arasında yazma süreleri elde etmiştir. Bu sonuçlar tutarlı fakat yüksek değerlerdir. Bu tutarlılık LanceDB'nin Apache Arrow mimarisinden kaynaklanmaktadır. Bu mimari, yazma hızı yerine okuma ve analitik sorgularda avantaj sunmaktadır. ChromaDB ise karşılaştırmadaki en yüksek yazma sürelerini üretmiştir. 83,043 - 97,134 saniye arasında yazma süreleri elde etmiştir. Bu sonuçlar ile ChromaDB, yazma işlemlerinde darboğaz oluşturmaktadır.

Bu benchmark sonucunda veri tabanı seçiminin kullanım senaryosuna bağlı olduğu görülmektedir. Hızlı prototipleme için ChromaDB kullanım kolaylığı sağlamaktadır. Yüksek yazma hızı ve ölçeklenebilirlik gerektiren senaryolar için Milvus ve Weaviate öne çıkmaktadır. Qdrant ise filtreleme ve hibrit arama gerektiren senaryolarda tercih edilebilir. LanceDB, sütun tabanlı Apache Arrow mimarisi sayesinde okuma ve analitik sorgularda avantaj sunmaktadır. Veri

tabanlarının yazma süresi performanslarının embedding modeline göre değişkenlik gösterdiği de göz önünde bulundurulmalıdır. Bu değişkenlik özellikle Qdrant'ta belirginleşmekte olup aynı veri tabanının farklı modeller altında birbirinden uzak sonuçlar ürettiği gözlemlenmiştir. Şekil 3'te sunulan nokta grafiği, 28 embedding modeli genelinde beş veri tabanının yazma süresi dağılımını görselleştirmekte ve veri tabanları arasındaki performans farklılıklarını model bazında karşılaştırmalı olarak ortaya koymaktadır.

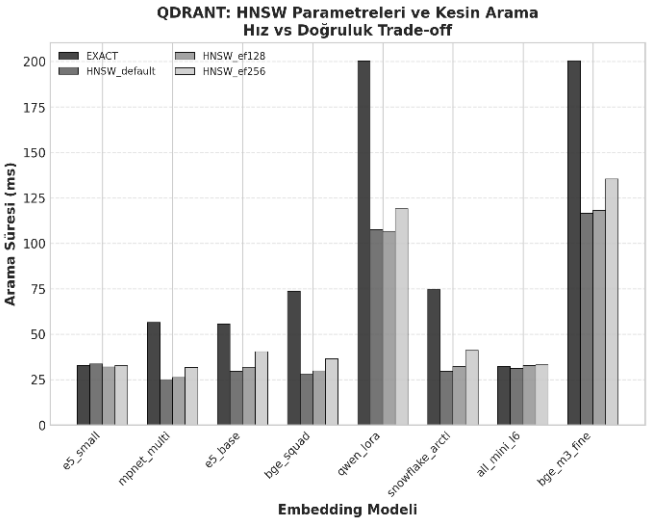


Şekil 3 Embeding modellerin veri tabanlarına yazma süreleri

A. Arama Stratejileri ve Parametre Varyasyonları

Geliştirilen RAG sisteminin belkemiğini oluşturan vektör veri tabanlarının (Milvus, Qdrant, ChromaDB, LanceDB ve Weaviate) getirme performanslarını objektif bir şekilde değerlendirebilmek için çoklu veri tabanı benchmark altyapısı kurulmuştur. Bu altyapı üzerinden, farklı veri boyutları ve donanım kısıtlamaları altında arama hızı ile getirim doğruluğu arasındaki optimum dengeyi bulmak amacıyla çeşitli arama stratejileri ve parametre varyasyonları test edilmiştir.

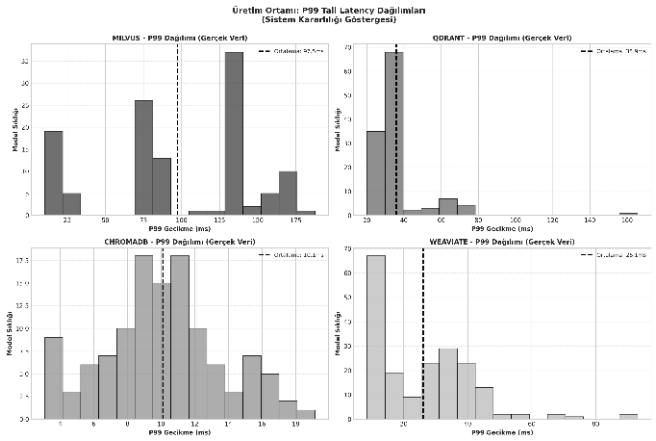
HNSW ve Yaklaşık Arama Varyasyonları Yüksek boyutlu veri setlerinde logaritmik arama hızları ve yüksek getirim doğruluğu sunan Grafik Tabanlı Hiyerarşik Gezinilebilir Küçük Dünya (HNSW) algoritması [20], projedeki birincil anlamsal arama stratejisi olarak belirlenmiştir. Qdrant üzerinde yapılan testlerde, HNSW'nin arama anındaki dinamik aday listesi boyutunu belirleyen hnsw_ef parametresi değiştirilerek hız ve doğruluk trade-off'u incelenmiştir [21]. Öte yandan, verileri kümelere ayırarak tarayan IVF (Inverted File) indeksleme yöntemini kullanan Milvus ve LanceDB sistemlerinde ise, sorgu esnasında taranacak alt küme sayısını ifade eden nprobe parametresi optimize edilerek arama uzayının maliyeti test edilmiştir. Kesin Arama, ANN algoritmalarının doğruluk kaybını ölçmek ve sistemin maksimum getirim tavanını belirlemek için, tüm vektör koleksiyonunun tek tek tarandığı kesin arama stratejisi uygulanmıştır. Qdrant üzerinde exact brute force parametresiyle gerçekleştirilen bu testler, %100 isabet oranına ulaşılmasını sağlarken, curse of dimensionality nedeniyle dramatik şekilde artan gecikme (latency) süreleri ve düşen saniyedeki sorgu sayısı (QPS) ile geleneksel HNSW indekslerinin performans avantajlarını Şekil 4'de net bir şekilde ortaya koymuştur [22]().



Şekil 4 Qdrant için farklı parametre yapılandırmalarında doğruluk-performans dengesi

Hibrit Sorgulama Vektör tabanlı anlamsal aramanın, ürün kodları veya özel kısaltmalar gibi tam kelime eşleşmesi gerektiren sözcüksel boşluk durumlarında yetersiz kalabildiği bilinmektedir. Bu sorunu çözmek için Weaviate veri tabanı üzerinde anlamsal near_vector araması ile geleneksel kelime tabanlı BM25 algoritmaları birleştirilerek hibrit arama senaryoları tasarlanmıştır [23]. Testler sırasında vektör ve metin skorlarının sentezlenme ağırlığını belirleyen alpha parametresi ayarlanarak optimum hibrit arama performansı değerlendirilmiştir.

Sistemin kararlılığını ölçmek ve önbellek ısınma sürelerinin test sonuçlarını yanıltmasını engellemek adına, projedeki değerlendirme metodolojisi 2 turluk ısınma (warmup) ve 5 turluk asıl test döngüsü şeklinde tasarlanmıştır. Arama stratejilerinin performansları ortalama, minimum ve maksimum sürelerin yanı sıra, şekil 5'te görüleceği üzere üretim ortamındaki gerçek performans darboğazlarını gösteren P50, P95, P99 yüzdelik gecikme metrikleri [24] ve toplam QPS (Query Per Second) üzerinden kayıt altına alınmıştır.

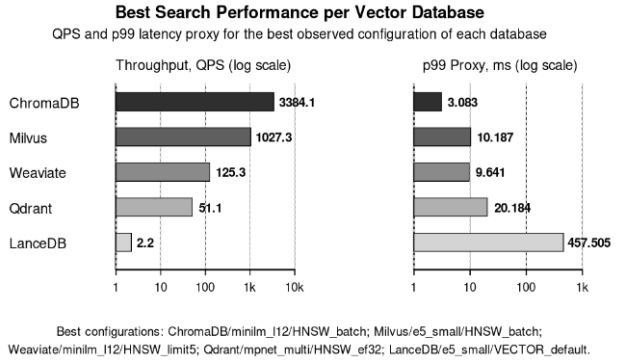


Şekil 5 Üretim ortamında P99 tail latency dağılımlarına göre sistem kararlılığının değerlendirilmesi

B. Performans Metrikleri

Bu bölümde vektör veri tabanı ve embedding modellerin performansını 4 metrik üzerinden değerlendirilmiştir. Bu metrikler saniyedeki sorgu sayısı (QPS), p99 gecikme, yazma hızı ve bellek tercih edilmiştir. Ölçümlerde 329 pdf belgesinden çıkarılan 52.331 metin parçası (vektör kaydı) kullanılmıştır. Milvus, Qdrant, ChormaDB, LanceDB ve Weaviate veri tabanları ile 28 farklı embedding model kullanılarak elde edilmiştir.

Arama performansında en yüksek QPS değeri ChoramaDB üzerinde minilm_l12 ve HNSW_batch algoritması ile ölçülmüştür. Bu ölçümde 3384,1 QPS değerine ulaşırken p99 gecikme vekili 3,083 ms olarak kaydedilmiştir.



Şekil 6 Vektör veritabanlarına göre en iyi arama performansı ve P99 gecikme karşılaştırması

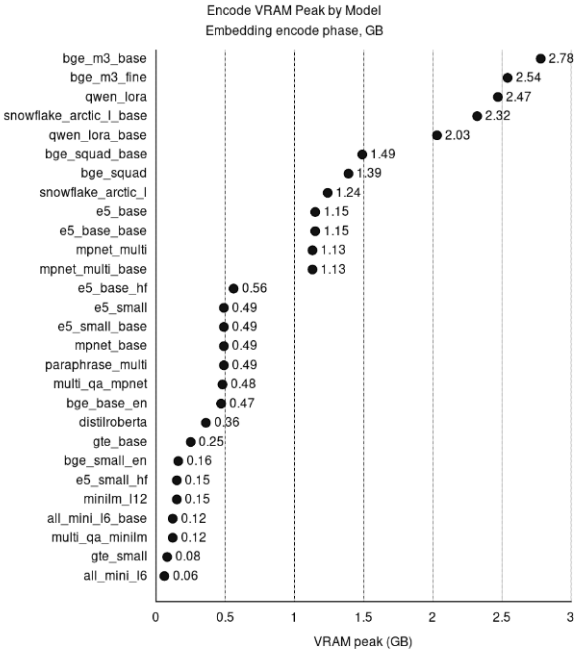
İlk on QPS sonucunun tamamı ChormaDB ve HNSW_batch kombinasyonundan oluşması, toplu sorgu işleme desteğinin işlem hacmi üzerinde belirgin bir avantaj sağladığını göstermektedir. Milvus, e5_small modeliyle 1027,3 QPS değerine ulaşarak ikinci en yüksek performansı üretmiştir. Weaviate ise minilm_l12 modeliyle 125,3 QPS ve 9,641 ms p99 gecikme değeriyle daha dengeli bir tekil sorgu profili sunmuştur.

Tablo 1'de detayları sunulan yazma hızı sonuçları, en hızlı tekil performans Milvus veri tabanı üzerinde elde edilmiştir. E5_small modeli ile 52.331 vektör kaydını 9.497 saniyede yazarak saniyede 5501,4 kayıt değerine ulaşmıştır. Bununla birlikte tüm modellerin ortalaması dikkate alındığında Weaviate saniyede 3484,1 kayıt ile daha tutarlı ve dengeli bir yazma performansı göstermektedir. ChromaDB ise saniyede 531,7 kayıt ortalama yazma hızıyla en düşük performansı sergilemiştir.

Write Performance Result					
DB	**N**	**Avg.**	**Median**	**Best Model**	**Max.**
Weaviate	28	3484,1	3569,6	minilm_l12	4841
Milvus	28	3235,7	3115,9	e5_small	5510,4
Qdrant	28	1585,5	1375,3	e5_small	2930,1
LanceDB	28	1187,4	1328,6	e5_small	1379,8
ChromaDB	28	531,7	578,1	e5_small	630,2

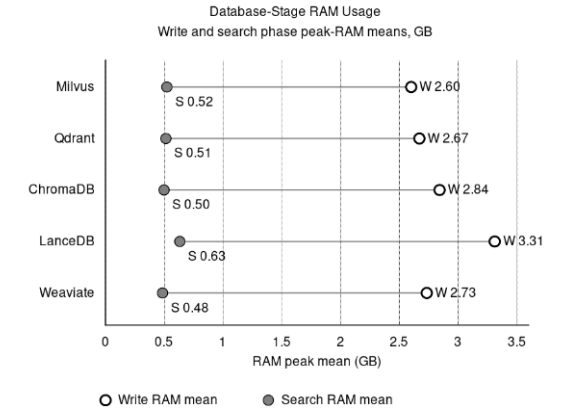
Tablo 1 Vektör veri tabanlarının yazma performanslarının karşılaştırmalı analizi ve optimum model konfigürasyonlarına göre elde edilen saniyedeki maksimum kayıt hızları.

Bellek kullanımı embedding encode fazı ve veri tabanı işlem fazı olmak üzere 2 ayrı fazda değerlendirilmiştir. Şekil 7'de detaylandırıldığı üzere, encode fazında en fazla VRAM kullanımı bge_m3_fine modelinde 2.54 GB olarak ölçülmüştür. snowflake_arctic_l_base 2,32 GB, qwen_lora_base ise 2,03 GB VRAM kullanımıyla yüksek bellek gerektiren modeller arasındadır. Buna karşılık all_mini_l6 0,06 GB, gte_small 0,08 GB ve multi_qa_minilm 0,12 GB VRAM kullanımıyla düşük kaynaklı sistemler için daha uygundur.



Şekil 7 Embedding modellerine göre encode aşamasındaki tepe VRAM kullanımı

Veri tabanı yazma ve arama aşamasında kaynak tüketimi ağırlık olarak RAM tarafında gözlemlenmiştir. Şekil 8'de görüleceği üzere, yüksek boyutlu modeller için yazma sırasında LanceDB üzerinde ortalama 3,310 GB ile en yüksek ram kullanımı, Qdrant üzerinde 2,669 GB ile en düşük kullanımı ölçülmüştür. Arama aşamasında ise LanceDB ve ChoramaDB yaklaşık 0.67 ve 0.75 GB aralığında olarak daha yüksek ram kullamını gösterirken, Weaviate ve Qdrant yaklaşık 440-446 MB aralığında kalarak daha düşük RAM kullanımı göstermiştir.



Şekil 8 Vektör veritabanlarına göre yazma ve arama aşamalarında ölçülen ortalama maksimum RAM kullanımı karşılaştırması

Genel olarak sonuçlar, performans gereksiniminin türüne göre farklı veri tabanı ve model seçimlerinin öne çıktığını göstermektedir. Aramada en yüksek sorgu işleme kapasitesi ChoramaDB üzerinde minilm_12 modeli ve HNSW_batch yapılandırmasıyla elde edilmiştir. Bu yapılanma 3384,1 QPS değerine ulaşırken p99 gecikme metriğinde yalnızca 3,083 ms düzeyinde kalmıştır. Milvus da e5_small modeliyle 1027.3 QPS değerine ulaşarak yüksek arama kapasitesine ulaşmıştır. Ancak ChromaDB'nin batch tabanlı yürütmesi, özellikle aynı anda birden fazla sorgunun işlendiği senaryolarda avantaj sağlamaktadır. Weaviate 125.3 QPS ve 9.641 ms p99 gecikme metriğiyle ChromaDb ve Milvus kadar yüksek performans vermese de daha dengeli bir tekli sorgu profili sunmaktadır. Qdrant ve LanceDB ise bu çalışmada arama kapasitesi bakımından sınırlı kalmıştır. Özellikler LanceDB'nin 2,2 QPS ve 457,505 ms p99 gecikme metrikleriyle, etkileşimli arama senaryoları için uygun değildir.

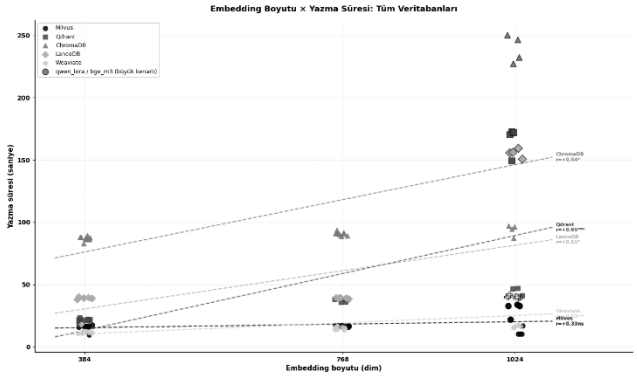
Yazma performansı açısından en yüksek tekil değer Milvus e5_small modeliyle 5510,4 kayıt/sn olarak görülmüştür. Ortalama yazma hızında ise Weaviate saniyede 3484,1 kayıt ile ilk sırada, Milvus saniyede 3235,7 kayıt ile ikinci sıradadır. Bellek kullanımında model seçimi özellikle encode fazında belirleyicidir. bge_m3_fine, snowflake_arctic_l_base ve qwe_lora_base yüksek vram gerektirirken, all_mini_l6, gre_small ve multi_qa_minilm düşük kayak tüketimi gözlemlenmiştir. Veri tabanı RAM tüketimi açısından Milvus ve Weaviate daha avantajlı görünmektedir.

Bu nedenle yüksek arama kapasitesi için ChromaDB, yüksek hızlı yazma için Milvus ve Weaviate, bellek kısıtlı ortamlarda ise düşük VRAM tüketimli embedding modelleri daha uygun seçeneklerdir.

V. DİSCUSSİON

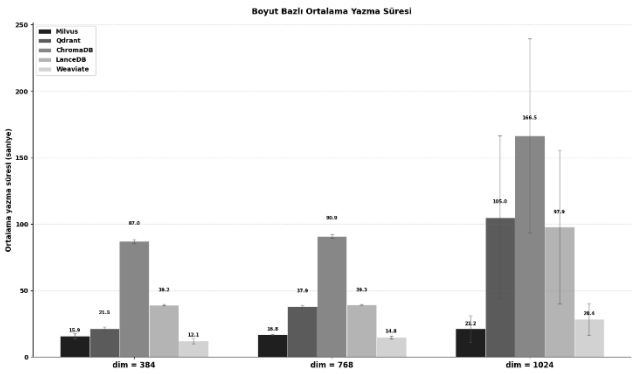
A. Yazma ve İndeksleme Performansı

Şekil 9'da sunulan karşılaştırma analizine göre embedding boyutu ile yazma süresi arasındaki ilişki veri tabanı mimarisine göre değişmektedir. Pearson korelasyon analizi, boyut artışının Qdrant, ChromaDB ve Weaviate üzerinde istatistiksel olarak pozitif bir etki yarattığını ortaya koymaktadır. LanceDB'de zayıf bir pozitif ilişki gözlemlenmektedir. Milvus'ta anlamlı bir ilişki bulunmamaktadır. Yani Milvus'da boyut artışı yazma süresine üzerine etki etmemektedir. Bu durum Milvus'un dağıtık indeksleme mimarisinin avantajını ortaya koymaktadır.



Şekil 9 Farklı vektör veritabanlarında embedding boyutunun yazma süresi üzerindeki etkisinin karşılaştırılması

Boyut gruplarına göre yazma süreleri incelendiğinde bu farklılaşma daha belirginleşmektedir. Şekil 10'da görüleceği üzere Qdrant'ta dim=384 için ortalama 21.5 saniye iken dim=1024'te 43.8 saniyeye yükselmektedir. Yani boyut üç katına çıktığında yazma süresi yaklaşık iki katına ulaşmaktadır. ChromaDB'de benzer bir örüntü gözlemlenmektedir. dim=384'ten dim=1024'e geçişte ortalama yazma süresi 87.0 saniyeden 93.9 saniyeye artmaktadır. Milvus ise dim=384'te 15.9, dim=768'de 16.8 ve dim=1024'te 11.9 saniye ortalama yazma süresiyle en tutarlı ve boyut bağımsız davranış profilini sergilemektedir. Qdrant'ta boyut-süre ilişkisi neredeyse doğrusal bir hat izlerken Milvus'ta bu ilişki görsel olarak da dağınık ve yönsüz kalmaktadır.



Şekil 10 Farklı embedding boyutlarında vektör veritabanlarının ortalama yazma süresi ve değişkenlik düzeyleri

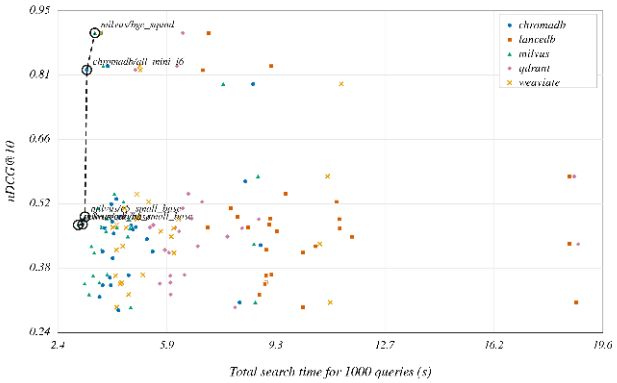
B. Arama Doğruluğu ve Hız Dengesi

Bu bölümde, 52.331 vektör kaydı üzerinde yürütülen benchmark çıktıları kullanılarak arama doğruluğu ile arama hızı arasındaki denge incelenmiştir. Doğruluk değerlendirmesi 1000 sorgu ve top_k=100 düzeninde nDCG@10, Recall@10/100, MRR@10 ve Hit@1/10 metrikleriyle yapılmıştır. Hız değerlendirmesinde aynı sorgu kümesinin toplam arama süresi ile algoritma bazlı QPS, ortalama gecikme ve p99_proxy değerleri kullanılmıştır.

Sonuçlarda, doğruluğunu temel olarak embedding model tarafından belirlendiği gözlemlenmiştir. Aynı doğruluk seviyesindeki hız farkınınsa veri tabanı ve arama algoritmasından kaynaklanmaktadır. En yüksek doğruluk tüm veri tabanlarında aynı skorları üreten bge_squad modeliyle elde edilmiştir. Bu skorlar nDCG@10=0,9008, Recall@10=0,970 ve MRR@10=0,8778 iken toplam sorgu süresi Milvus üzerinde 3,585 s iken LanceDB üzerinde 7,173 s olarak ölçülmüştür. Tablo 2'de detayları verilen bu değerler, modelin sabit tutulduğunda gecikmenin asıl olarak veri tabanından kaynaklandığını göstermektedir. Vektör veri tabanlarının bge_squad modeli altındaki gecikme performansı ve toplam arama süresi ilişkisi Şekil 11'de görselleştirilmiştir.

DB	Model	Dim	nDCG@10	Recall@10	1000 queries (s)	QPS
milvus	bge_squad	1024	0.9008	0.970	3.585	278.9
weaviate	bge_squad	1024	0.9008	0.970	3.762	265.8
chromadb	bge_squad	1024	0.9008	0.970	3.786	264.1
qdrant	bge_squad	1024	0.9008	0.970	6.356	157.3
lancedb	bge_squad	1024	0.9008	0.970	7.173	139.4

Tablo 2 bge_squad modeli için aynı doğruluk düzeyinde veritabanı gecikme değerleri ve arama süresi–nDCG@10 ilişkisi



Şekil 11 bge_ modeli altında vektör veritabanlarının aynı doğruluk seviyesindeki gecikme performansı ve toplam arama süresine göre nDCG@10 dağılımı

Yukarıda yer alan grafikteki performans-doğruluk dengesi noktaları, daha hızlı ve daha doğru bir yapılandırma tarafından domine edilmeyen kombinasyonları göstermektedir. En hızlı denge noktası Milvus/gte_base olsa da bu yapılandırma yalnızca 0,4748 nDCG@10 değerine ulaşmıştır. ChromaDB/all_mini_l6 kombinasyonu en dengeli ara noktadır. Bu nokta 3,330 saniye toplam arama süresinde 0,8188 nDCG@10 ve 0,928 Recall@10 üretmiştir. Milvus/bge_squad ise 3,585 saniyeyle ufak bir ek gecikme karşılığında 0,9008 nDCG@10 seviyesine çıkarak doğruluk odaklı en güçlü performans-doğruluk dengesi noktasını oluşturmuştur. Söz konusu denge noktaları ve Pareto cephesinde yer alan en verimli yapılandırmalar Tablo 3'de sunulmuştur.

DB	Model	Dim	nDCG@10	Recall@10	1000 queries (s)	QPS
milvus	gte_base	768	0.4748	0.594	3.056	327.2
chromadb	e5_small_base	384	0.4761	0.599	3.196	312.9
milvus	e5_small_base	384	0.4929	0.622	3.270	305.8
chromadb	all_mini_l6	384	0.8188	0.928	3.330	300.3
milvus	bge_squad	1024	0.9008	0.970	3.585	278.9

Tablo 3 nDCG@10 ve toplam arama süresi ölçütlerine göre Pareto cephesinde yer alan en dengeli yapılandırmalar

Algoritma bazlı sorgu işleme kapasitesi sonuçları, özellikle toplu HNSW sorgulamanın başarısını göstermektedir. Yüksek doğruluk eşiği korunurken ChromaDB/bge_squad HNSW_batch yapılandırması 1331,4 QPS ve 9,429 ms p99_proxy ile en yüksek semantik arama sorgu işleme kapasitesi değerini üretmiştir. Aynı model Milvus üzerinde

423,8 QPS, Weaviate üzerinde 82,3 QPS, Qdrant üzerinde 43,9 QPS ve LanceDB üzerinde 2,1 QPS düzeyindedir**.** Tablo 4'de görüleceği üzere bu nedenle tek kullanıcı senaryolarında Milvus/bge_squad güçlü bir doğruluk gecikme oranı sunarken çok kullanıcılı veya toplu sorgu senaryolarında ChromaDB/HNSW_batch daha yüksek sorgu işleme kapasitesi üretmektedir.

DB	Algorithm	nDCG@10	QPS	Avg. ms	p99_proxy ms
chromadb	HNSW_batch	0.9008	1331.4	7.511	9.429
milvus	HNSW_batch	0.9008	423.8	23.598	29.519
weaviate	HNSW_limit5	0.9008	82.3	12.155	13.131
qdrant	HNSW_ef32	0.9008	43.9	22.791	23.244
lancedb	VECTOR_default	0.9008	2.Oca	483.725	491.438

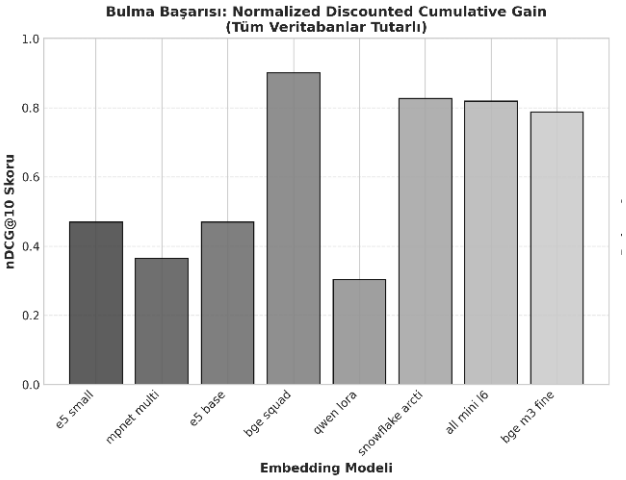
Tablo 4 nDCG@10 değeri sabit tutulan yüksek doğruluklu yapılandırmalarda algoritma bazlı QPS, ortalama gecikme ve P99 gecikme karşılaştırması

Bulgular en iyi yapılandırma olmadığını, duruma göre farklılık gösterdiği gözlemlenmiştir. Doğruluğun kritik olduğu akademik, hukuki veya kurumsal senaryolarda bge_squad tercih edilmelidir. Düşük kaynaklı kurumlarda ise all_mini_l6 modeli, 384 boyutlu vektörleriyle az maliyeti ve 0,8188 nDCG@10 düzeyi sayesinde pratik bir denge sağlar. Maksimum sorgu işleme kapasitesi hedefinde ise ChromaDB/HNSW_batch öne çıkmaktadır.

C. Gömme Modellerinin Karşılaştırmalı Analizi

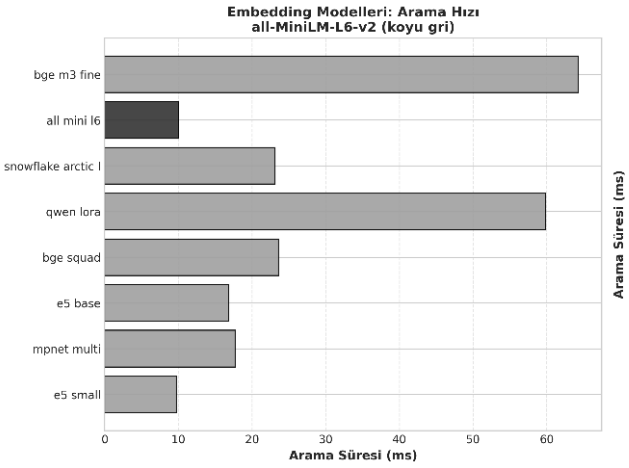
Yerel RAG mimarisinde anlamsal eşleştirmenin başarısı, doğrudan embedding modellerinin yapılandırılmamış metinleri yüksek boyutlu vektör uzayında ne kadar doğru hizaladığına ve kümelediğine bağlıdır. Bu çalışmada, all-MiniLM-L6-v2, BGE [25] ve GTE türevleri gibi farklı açık kaynaklı dil modellerinin vektör uzayındaki anlamsal temsil ve hizalanma yetenekleri karşılaştırmalı olarak analiz edilmiştir. Literatürdeki MTEB (Massive Text Embedding Benchmark) gibi geniş çaplı değerlendirmeler de göstermektedir ki, modelin parametre boyutunun artması her zaman her görevde doğrusal bir performans artışı anlamına gelmemektedir kompakt modeller de belirli görevlerde oldukça rekabetçi olabilmektedir [26].

Elde ettiğimiz bulgular, all-MiniLM-L6-v2 modelinin bilgi damıtma mimarisine sahip olmasına rağmen kurumsal metinlerdeki kavramsal ilişkileri vektör uzayında yüksek doğrulukla kümeleyebildiğini ve eş anlamlılık/bağlam sorunlarını başarıyla çözdüğünü göstermiştir.



Şekil 11 Farklı gömme modellerinin tüm veritabanları genelinde nDCG@10 metriğine göre erişim başarımı

Modellerin anlamsal hizalanma başarısının yanı sıra, bu modellerin üretim ortamındaki pratik karşılığını ve gerçek zamanlı performansını ölçmek, sistemin uygulanabilirliği açısından hayati önem taşımaktadır. Özellikle kısıtlı donanım kaynaklarına sahip sistemlerde uygulanan modellerin yalnızca gerektiğinde GPU'ya çekilip ardından belleğin temizlenmesi [27], vektörizasyon aşamasında ciddi bir darboğaz yaratma potansiyeline sahiptir.



Şekil 12 Farklı gömme modellerinin arama süresi bakımından karşılaştırmalı performansı

Bu donanımsal kısıtlamanın etkilerini doğru analiz edebilmek için, ortalama hızlardan ziyade, sistem kararlılığını ve uç durumları gösteren P95 gecikme metrikleri kullanılmıştır. P95 metriği, sistemin en yoğun eşzamanlı sorgu anlarında bile taleplerin %95'ini hangi maksimum sürede vektörize edip işleyebildiğini kanıtlamaktadır.

Yapılan testlerde, daha yüksek parametre sayısına sahip modellerin (örneğin BGE M3) vektör uzayında nispeten daha hassas bir ayrışma sunmasına karşın, CPU'dan GPU'ya veri aktarımı ve bellek tahsisi sırasında P95 gecikme sürelerinde dramatik sıçramalara (latency spikes) neden olduğu gözlemlenmiştir [27]. Buna karşın, sistemde varsayılan

olarak seçilen all-MiniLM-L6-v2, kompakt yapısı sayesinde dinamik bellek yönetimi (swapping) altında bile son derece kararlı ve düşük P95 gecikme süreleri sunmuştur. Sonuç olarak, etkileşimli bir asistan için en kritik faktörlerden biri olan kabul edilebilir yanıt süresi, MiniLM modeliyle güvence altına alınmış [28] anlamsal geri getirim doğruluğu (recall) ile P95 uç gecikme maliyetleri arasında en ideal denge kurulmuştur.

VI. Conclusions

Bu çalışmada, bilgi güvenliğini tehdit eden bulut tabanlı LLM sistemlerine alternatif olarak, tamamen lokalde çalışan yüksek performanslı RAG sistemi tasarlanmıştır. Ve kapsamlı bir performans analizi sunulmuştur. Geleneksel anahtar tabanlı arama algoritmaları eksik kalan kısımlarını embedding katmanı ve vektör veri tabanı kullanıp anlamsal eşleştirme yaparak çözmüştür. Elde edilen sonuçlarda, kısıtlı kaynaklarda bile dinamik bellek yönetimi ve doğru embedding model – vektör veri tabanı yapılandırması sayesinde yüksek başarı sunabilmiştir. Bu mimari sağlık, savunma sanayi ve finans gibi veri gizliliği önemli olan sektörlerde derin araştırma yapabilen akıllı asistanların kullanılabilirliğini bilimsel olarak ortaya koymaktadır.

Araştırmanın literatüre ve endüstriye sunduğu en kritik katkılardan bir diğeri ise, farklı vektör veri tabanları ve embedding modelleri arasındaki hız, doğruluk ve donanım maliyeti dengesini ortaya koyan benchmark altyapısıdır. Test sonuçları, RAG sistemlerinde tek bir en iyi çözümün olmadığını göstermiştir. Yüksek veri yazma hızı gerektiren durumlarda Milvus'un, maksimum toplu sorgu kapasitesi hedeflendiğinde ChromaDB'nin, yüksek doğruluk arayışında ise bge_squad modelinin öne çıktığını gözlemlenmiştir. Sonuç olarak bu çalışma LLM'lerin lokal ortamda güvenli, şeffaf ve izlenebilir bir şekilde entegre edilebileceğini kanıtlamıştır. Bilgiye erişim süreçlerini optimize ederek verimlilik artışı vaat etmektedir.

VII. References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, *33*, 9459-9474.

[2] Robertson, S., & Zaragoza, H. (2009). *The probabilistic relevance framework: BM25 and beyond* (Vol. 4). Now Publishers Inc.

[3] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, *2*(1), 32.

[4] Gao, Y., Xiong, Y., Wang, M., & Wang, H. (2024). Modular rag: Transforming rag systems into lego-like reconfigurable frameworks. *arXiv preprint arXiv:2407.21059*.

[5] Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021, November). Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021* (pp. 3784-3803).

[6] Rackauckas, Z. (2024). Rag-fusion: a new take on retrieval-augmented generation. *arXiv preprint arXiv:2402.03367*.

[7] Gao, L., Ma, X., Lin, J., & Callan, J. (2023, July). Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 1762-1777).

[8] Clough, P., & Sanderson, M. (2013). Evaluating the performance of information retrieval systems using test collections.

[9] Oche, A. J., Folashade, A. G., Ghosal, T., & Biswas, A. (2025). A systematic review of key retrieval-augmented generation (rag) systems: Progress, gaps, and future directions. *arXiv preprint arXiv:2507.18910*.

[10] Xu, Z., Mo, F., Huang, Z., Zhang, C., Yu, P., Wang, B., ... & Srikumar, V. (2025). A survey of model architectures in information retrieval. *arXiv preprint arXiv:2502.14822*.

[11] Trappolini, G., Cuconasu, F., Filice, S., Maarek, Y., & Silvestri, F. (2026, March). Redefining retrieval evaluation in the era of llms. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 8359-8375).

[12] Khattab, O., & Zaharia, M. (2020, July). Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval* (pp. 39-48).

[13] Honovich, O., Aharoni, R., Herzig, J., Taitelbaum, H., Kukliansy, D., Cohen, V., ... & Matias, Y. (2022, July). TRUE: Re-evaluating factual consistency evaluation. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 3905-3920).

[14] Agrawal, A., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., & Ramjee, R. (2023). Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*.

[15] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., ... & Stoica, I. (2023, October). Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles* (pp. 611-626).

[16] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., ... & Stoica, I. (2023, October). Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles* (pp. 611-626).

[17] Song, M., & Zheng, M. (2024). A survey of query optimization in large language models. *arXiv preprint arXiv:2412.17558*.

[18] Rahmani, H. A., Wang, X., Feng, Y., Zhang, Q., Yilmaz, E., & Lipani, A. (2023, July). A survey on asking clarification questions datasets in conversational systems. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 2698-2716).

[19] Cao, Z., Li, P., & Zhu, Q. (2025, November). ICR: Iterative Clarification and Rewriting for Conversational Search. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (pp. 9821-9835).

[20] Es, S., James, J., Anke, L. E., & Schockaert, S. (2024, March). Ragas: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th conference of the european chapter of the association for computational linguistics: system demonstrations* (pp. 150-158).

[21] Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, *42*(4), 824-836.

[22] Aumüller, M., Bernhardsson, E., & Faithfull, A. (2020). ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, *87*, 101374.

[23] Jain, N., Chiang, P. Y., Wen, Y., Kirchenbauer, J., Chu, H. M., Somepalli, G., ... & Goldstein, T. (2024, May). Neftune: Noisy embeddings improve instruction finetuning. In *International Conference on Learning Representations* (Vol. 2024, pp. 17566-17591).

[24] Aggarwal, C. C., Hinneburg, A., & Keim, D. A. (2001, January). On the surprising behavior of distance metrics in high dimensional space. In *International conference on database theory* (pp. 420-434). Berlin, Heidelberg: Springer Berlin Heidelberg.

[25] Li, Y. W., Li, L. J., Song, X. K., & Wang, D. (2022). Trade-off relations of quantum resource theory in neutrino oscillations. *The European Physical Journal Plus*, *137*(11), 1267.

[26] Dean, J., & Barroso, L. A. (2013). The tail at scale. Communications of the ACM, 56(2), 74–80.

[27] Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024). BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. arXiv:2402.03216.

[28] Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive text embedding benchmark. In Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (pp. 2014–2037). ACL.